

Тема 1. Введение в системное программирование

Системное программирование — это процесс разработки программного обеспечения, которое взаимодействует с аппаратным обеспечением и операционной системой на низком уровне. Это включает написание кода, который работает непосредственно с ресурсами компьютера, такими как процессор, память, устройства ввода-вывода и сети. В отличие от прикладного программного обеспечения, которое ориентировано на конечного пользователя, системное программное обеспечение (СПО) предназначено для обеспечения функционирования компьютерной системы и других программ.

Примеры системного программирования:

1. Операционная система (ОС)

Операционная система (например, Windows, Linux, macOS) — это главный пример системного программного обеспечения. Она управляет всеми ресурсами компьютера: процессором, памятью, жестким диском, устройствами ввода-вывода (клавиатурой, мышью, принтером).

Пример: когда ты открываешь программу, ОС выделяет ей память и процессорное время.

Пример: когда ты подключаешь флешку, ОС распознает устройство и позволяет тебе с ним работать.

2. Драйверы устройств

Драйверы — это программы, которые позволяют операционной системе взаимодействовать с аппаратным обеспечением (например, видеокартой, принтером, сетевой картой).

Пример: Когда ты печатаешь документ, драйвер принтера передает данные из программы (например, Word) на принтер, чтобы он мог их распечатать.

3. Файловые системы

Системные программы управляют тем, как данные хранятся на диске. Они отвечают за создание, удаление и организацию файлов.

Пример: Когда ты сохраняешь файл на жесткий диск, операционная система решает, где именно он будет храниться и как его можно будет найти.

4. Сетевые протоколы

Системные программы обеспечивают работу сети, управляя передачей данных между компьютерами.

Пример: Когда ты открываешь сайт в браузере, системное программное обеспечение отвечает за отправку и получение данных через интернет.

5. Управление памятью

Системные программы распределяют память между запущенными приложениями и следят, чтобы одна программа не "залезла" в память другой.

Пример: Если ты одновременно открываешь браузер и игру, операционная система выделяет каждому из них свою часть оперативной памяти.

Чем системное программирование отличается от прикладного?

Прикладное программирование — это создание программ для пользователей (например, текстовые редакторы, игры, мессенджеры). Эти программы работают "поверх" операционной системы и используют её ресурсы.

Системное программирование — это создание программ, которые управляют самими ресурсами компьютера и обеспечивают работу прикладных программ.

Тема 2. введение в программирование

Компьютерная программа — список команд (инструкций) для компьютера. Команды могут быть любыми, например:

- считать информацию с клавиатуры;
- произвести арифметические вычисления (+, −, *, /);
- вывести информацию на экран.

Язык программирования

Язык программирования — набор определенных правил, по которым компьютер может понимать команды (инструкции) и выполнять их. Текст программы на любом языке программирования называется программным кодом.

Языки программирования бывают компилируемые и интерпретируемые. Если программа написана на компилируемом языке (C, C++, Pascal), то перед выполнением её нужно полностью проверить на наличие синтаксических ошибок и уже после этого перевести в понятную для компьютера форму — машинный код. Это делает специальная программа, которая называется компилятором.

Если программа написана на интерпретируемом языке (Python, PHP, Ruby), она не переводится в машинный код целиком. Вместо этого специальная программа, которая называется интерпретатором, идет по коду, анализирует его и выполняет каждую отдельную команду.

Существуют языки программирования, которые совмещают оба подхода (C#, Java). В таких языках код исходной программы сначала компилируется в промежуточный код (байт-код), а уже потом, во время выполнения, переводится в машинный код.

Язык Python

Язык Python разработал голландский программист Гвидо Ван Россум (Guido van Rossum) в 1991 году. Гвидо был фанатом британского комедийного сериала «Monty Python's Flying Circus», откуда и пришло название языка.

Преимущества Python

- Это интерпретируемый язык программирования: он не требует отдельного этапа компиляции; программа на языке Python запускается прямо из исходного кода;
- Это высокоуровневый язык программирования;

– Это платформонезависимый язык: программы на Python можно создавать на разных операционных системах (Linux, Windows, OS X); программы на Python можно запускать на разных операционных системах (Linux, Windows, OS X);

- Это open source проект;
- Это простой язык;
- Это встраиваемый скриптовый язык;
- Это динамический язык, что упрощает написание несложных программ;
- Для Python существует огромная библиотека классов на любой вкус.

Недостатки Python

- Низкая скорость выполнения по сравнению с такими языками, как C и C++;
- Динамическая типизация языка — минус при написании сложных программ.

Тема 2.1. Аргументы команды `print()`

То, что мы пишем в круглых скобках у команды `print()`, называется **аргументами** команды.



Аргументы – это конкретные значения, которые вы передаете функции при ее вызове.

Команда `print()` позволяет указывать несколько аргументов, в таком случае **их надо отделять запятыми**.

Приведённый ниже код:

```
print('Я', 'учусь', 'программировать', 'на', 'Python!')
```

ВЫВОДИТ:

```
Я учусь программировать на Python!
```

Обратите внимание, в качестве разделителя при выводе данных между аргументами команды используется **символ пробела**. По умолчанию команда `print()` добавляет **ровно один пробел** между всеми своими аргументами.

Приведённый ниже код:

```
print('1', '2', '4', '8', '16')
```

ВЫВОДИТ:

```
1 2 4 8 16
```



Запомни: при написании кода между аргументами команды

`print()` после запятой мы ставим один символ пробел. Это

общепринятое соглашение в языке Python. Этот символ пробела не влияет на вывод данных. Это просто **для читаемости кода**.

Примечания

Примечание 1. Команда `print()` записывается только маленькими буквами, другое написание недопустимо, так как в Python строчные и заглавные буквы [различны](#).

Примечание 2. Каждая последующая команда `print()` выводит указанный текст **с новой строки**.

Приведённый ниже код:

```
print('Какой хороший день!')  
print('Работать мне не лень!')
```

выводит:

```
Какой хороший день!  
Работать мне не лень!
```

Примечание 3. Команда `print()` с пустым списком аргументов просто вставляет новую пустую строку.

Приведённый ниже код:

```
print('Какой хороший день!')  
print()  
print('Работать мне не лень!')
```

выводит:

```
Какой хороший день!  
  
Работать мне не лень!
```

Обратите внимание на то, что вторая строка пустая.

Примечание 4. Почему в Python можно использовать как одинарные, так и двойные кавычки для обрамления текста? Ответ очень прост — часто кавычки являются частью текста. И чтобы Python мог правильно распознать такой текст, пользуемся разными:

- если в тексте нужны одинарные кавычки, то для обрамления такого текста используем двойные кавычки;
- если в тексте нужны двойные кавычки, то обрамляем его одинарными.

Приведённый ниже код:

```
print('В тексте есть "двойные" кавычки')  
print("В тексте есть 'одинарные' кавычки")
```

Выводит:

```
В тексте есть "двойные" кавычки  
В тексте есть 'одинарные' кавычки
```

Примечание 5. Обратите внимание, что в одном `print()` мы можем комбинировать одинарные и двойные кавычки. Это делает наш код более гибким, позволяя легко включать кавычки и апострофы в наши строки.

Приведённый ниже код:

```
print("I'm", 'the', "BAD", 'guy')
```

Выводит:

```
I'm the BAD guy
```

Python не делает различий между одинарными и двойными кавычками: они работают одинаково для определения строк.

Примечание 6. Мы не можем использовать одинарные кавычки в строке, если саму строку обрамляем одинарными кавычками. С двойными кавычками ситуация аналогичная.

Приведённый ниже код:

```
print('12 ' 34')
```

приводит к возникновению ошибки:

```
SyntaxError: unterminated string literal (detected at line 1)
```

1. Отсутствие кавычек `' '` для строк в команде `print()` .

✗ Неправильно:

```
print(Я также люблю математику !)
```

и

```
print(Я, также, люблю, математику, !)
```

✓ Правильно:

```
print('Я также люблю математику !')
```

и

```
print('Я', 'также', 'люблю', 'математику', '!')
```

В Python строки всегда должны быть заключены в кавычки, иначе возникнет ошибка.

2. Знак `=` после команды `print()` :

✗ Неправильно:

```
print = ('Оппенгеймер vs Барби')
```

✓ Правильно:

```
print('Оппенгеймер vs Барби')
```

Ввод данных, команда `input()`, переменные

Все предыдущие программы выводили на экран текст, известный в момент написания программного кода. Однако программы могут работать с данными, которые станут известны только во время выполнения программы. Другими словами, программы могут **считывать данные**, а затем их использовать.

Для считывания данных в языке Python используется команда `input()` .

Рассмотрим следующую программу:

```
print('Как тебя зовут?')      # вывод текста
print('Привет,', input())    # ввод текста и вывод текста
```

Сначала программа распечатает текст на экран `'Как тебя зовут?'` .

Далее программа будет ждать от пользователя ввода данных. Ввод данных реализуется с помощью команды `input()` . И в конце произойдет вывод текста вместе с введенными от пользователя данными.

Команда `input()` всегда пишется с круглыми скобками. Она работает так: когда программа доходит до места, где есть `input()` , она ждет, пока пользователь введет текст с клавиатуры (ввод завершается нажатием клавиши **Enter**). Введенная строка подставляется на место `input()` .

Если вы используете IDE VS Code, то у вас ввод будет требоваться в окне **TERMINAL**. Сюда и нужно вводить текст:

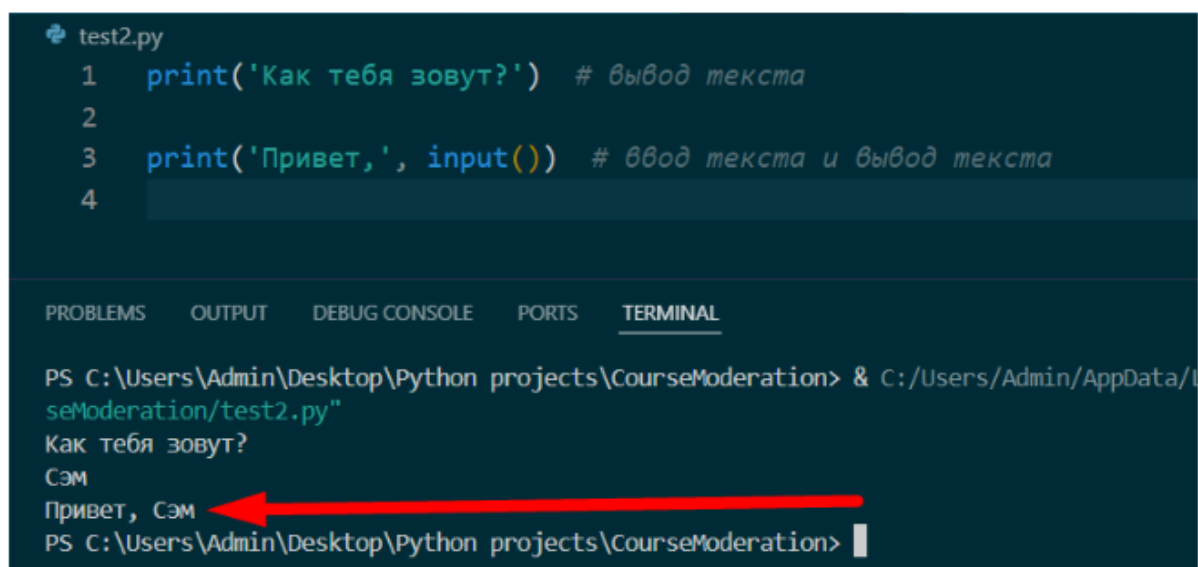


```
test2.py
1 print('Как тебя зовут?') # вывод текста
2
3 print('Привет,', input()) # ввод текста и вывод текста
4

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

PS C:\Users\Admin\Desktop\Python projects\CourseModeration> & C:/Users/Admin/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe C:/Users/Admin/Desktop/CourseModeration/test2.py
Как тебя зовут?
```

В VS Code после введения текста нужно нажать клавишу **Enter**:



```
test2.py
1 print('Как тебя зовут?') # вывод текста
2
3 print('Привет,', input()) # ввод текста и вывод текста
4

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

PS C:\Users\Admin\Desktop\Python projects\CourseModeration> & C:/Users/Admin/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe C:/Users/Admin/Desktop/CourseModeration/test2.py
Как тебя зовут?
Сэм
Привет, Сэм
PS C:\Users\Admin\Desktop\Python projects\CourseModeration>
```

После нажатия клавиши **Enter** у нас был выведен текст `'Привет, Сэм'`, и программа завершила свою работу.

Если вы ввели строку `Сэм`, программа дальше работала так же, словно на месте `input()` было написано `'Сэм'`.

Таким образом, команда `input()` получает от пользователя (нас) какие-то данные и вместо вызова подставляет **строковое значение**.

Запоминаем:



Команда `print()` **выводит данные** на экран.

Команда `input()` **считывает данные**, введенные с клавиатуры.

Переменные

Команда `input()` обозначает: "подожди, пока пользователь введет что-нибудь с клавиатуры, и запомни то, что он ввел". Просто так просить "запомнить" довольно бессмысленно: нам ведь потом надо будет как-то сказать компьютеру, чтобы он вспомнил то, что запомнил! Для этого мы используем **переменные** и пишем следующий код:

```
variable_name = input()
print('Вы ввели текст:', variable_name)
```

Данный код означает: "Сохрани то, что ввел пользователь, в памяти, и дальше это место в памяти мы будем называть именем `variable_name`". Соответственно, команда `print(variable_name)` означает: "Посмотри, что лежит в памяти под именем `variable_name`, и выведи это на экран".

Вот такие "места в памяти" и называются переменными. Любая переменная имеет **имя и значение**.

Переменные в команде print()

Как вы заметили, мы можем использовать переменные в команде `print()`. Для этого мы просто указываем имя переменной без кавычек внутри круглых скобок как аргумент.

Приведенный ниже код:

```
city = 'Тула'
print(city, '- мой город!')
```

выводит:

```
Тула - мой город!
```

Мы можем использовать несколько переменных в команде `print()`, нам это никто не запрещает. Только стоит помнить, что аргументы в команде `print()` **нужно отделять запятыми** друг от друга.

Приведенный ниже код:

```
name = 'Алеша'
city = 'Тула'
print('Меня зовут', name, '.', city, '- мой город!')
```

выводит:

```
Меня зовут Алеша . Тула - мой город!
```


Имя переменной

1. В имени переменной рекомендуется использовать только латинские буквы `a-z`, `A-Z`, цифры и символ нижнего подчеркивания (`_`)
2. Имя переменной не может начинаться с цифры
3. Имя переменной не может содержать пробелы
4. Имя переменной по возможности должно отражать ее назначение



Запомни: Python – регистрочувствительный язык. Переменные `name` и `Name` – две совершенно разные переменные.

Для именования переменных принято использовать стиль

`lower_case_with_underscores` (слова из маленьких букв с подчеркиваниями).

Например, следующие имена для переменных хорошие:

```
server_response
is_password_good
username
cyberpunk_77
```

Значение переменной

Значение переменной — это сохранённая в ней информация: текст, число и т.д.

Символ `=` в языках программирования выступает в роли **оператора присваивания**.

Он выполняет задачу связывания значения, расположенного справа от него, с именем переменной, указанной слева. Таким образом, переменная слева

"принимает" или "хранит" значение, представленное выражением справа от знака `=`:

имя переменной = значение переменной



Запомни: интерпретатор ждет, что пользователь что-то введет с клавиатуры ровно столько раз, сколько команд `input()` встречается в программе. Каждый `input()` завершается нажатием **Enter** на клавиатуре.

Значения переменной, естественно, можно менять (переприсваивать).

```
print('Как тебя зовут?')

name = input()
print('Привет,', name)

name = 'Timur'
print('Привет,', name)
```

Оператор присваивания сообщает переменной то или иное значение независимо от того, была ли эта переменная введена раньше. Вы можете менять значение переменной, записав еще один оператор присваивания. Если у нас имеется переменная, мы можем делать с ней все что угодно — например, присвоить другой переменной или изменить значение.

Приведённый ниже код:

```
name1 = 'Тимур'
name2 = name1
name1 = 'Гвидо'

print(name1)
print(name2)
```

ВЫВОДИТ:

```
Гвидо
Тимур
```



Запомни: название переменной всегда должно быть **слева от знака равенства**.

Приведённый ниже код:

```
'Timur' = name
```

приводит к возникновению ошибки:

```
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='?
```

Примечания

Примечание 1. Очень часто перед считыванием данных мы печатаем некоторый текст, чтобы пользователь, который вводит эти данные, понимал, что именно от него требуется. Например, в программе:

```
print('Как тебя зовут?')
name = input()
```

Мы сначала выведем текст "Как тебя зовут?", а уже потом считаем данные.

Поскольку это достаточно распространенный сценарий, то в языке Python можно выводить текст, передавая его в качестве параметра в команду `input()`.

Предыдущий код можно переписать так:

```
# сначала тут печатается строка 'Как тебя зовут', а потом принимается на
вход имя
name = input('Как тебя зовут?')
```

То есть команда `input()` при наличии аргументов внутри нее отрабатывает одновременно как вывод текста, а потом ввод текста (именно в этом порядке). Однако в задачах нашего курса нужно использовать команду `input()` **без параметров!** Так как это будет лишний вывод, ваша программа не будет проходить тесты.

Примечание 2. Имейте в виду, что мы можем принимать сразу несколько строк, а потом со всеми ними работать.

```
first = input()
second = input()

print('I am', first, 'and', second)
```

Например, если на вход программе будут поданы строки:

```
cool
I know it
```

то она выведет следующее:

```
I am cool and I know it
```

Примечание 3. Переменные можно вводить в любой момент (не только в самом начале программы):

```
name_1 = input()                # принимаем имя первого
человека                        # делаем первый вывод
print('Привет,', name_1, '!')

name_2 = input()                # принимаем имя второго
человека                        # делаем второй вывод
print('Здравствуйте,', name_2, '.')
```

Например, если на вход программе будут поданы строки:

```
Билл
Тед
```

то она выведет следующее:

```
Привет, Билл !
Здравствуйте, Тед .
```

1. Использование текста или других аргументов в команде `input()` при считывании данных:

 **Неправильно:**

```
name = input('Введите свое имя:')
```

 **Правильно:**

```
name = input()
```

Хотя писать текст внутри команды `input()` допускается и иногда даже рекомендуется в программировании, так **нельзя делать** при решении наших задач, так как это произведёт лишний вывод в консоль, и ваша программа не пройдёт тесты.

2. Использование пробелов в названии переменных:

✗ Неправильно:

```
my name = input()
```

✓ Правильно:

```
my_name = input()
```

3. Оборачивание переменной в кавычки:

✗ Неправильно:

```
name = input()
print('Меня зовут', 'name')
```

В данном случае вы выводите не само значения переменной `name`, а именно текст «name».

✓ Правильно:

```
name = input()
print('Меня зовут', name)
```

4. Обработка частного случая, решение не в общем виде – отсутствие приёма входных данных через `input()`:

✗ Неправильно:

```
print('Моего кота зовут', 'Томас')
```

✓ Правильно:

```
cat_name = input()
print('Моего кота зовут', cat_name)
```

Ваша программа должна решать задачу в общем виде. Мы изначально не знаем, какие входные данные поступают на вход, поэтому мы принимаем их через команду `input()` и записываем это в переменную. Далее эту переменную мы используем в программе для соответствующего вывода.

5. Отсутствие круглых скобок для команды `input()` :

✗ Неправильно:

```
name = input
age = input

print('Мое имя -', name, 'и мне', age, 'лет!')
```

✓ Правильно:

```
name = input()
age = input()

print('Мое имя -', name, 'и мне', age, 'лет!')
```

Если вы не будете писать круглые скобки, то команда `input()` не будет вызвана, и при выводе переменных `name` и `age` вы будете видеть текст `<built-in function input>`. Этот текст является лишь строковым представлением команды `input()` в Python.

6. Отсутствие закрывающей скобки для команды `input()` :

✗ Неправильно:

```
city = input(
print('Я живу в городе', city)
```

✓ Правильно:

```
city = input()
print('Я живу в городе', city)
```

Для вызова команды `input()` нужны две скобки – открывающая и закрывающая. Если скобок не будет хватать, Python воспримет это как ошибку синтаксиса, и ваш код не сработает.

Необязательные параметры команды print

По умолчанию команда `print()` принимает несколько аргументов, выводит их через **один пробел**, после чего **ставит перевод строки**. Это поведение можно изменить, используя необязательные именованные параметры `sep` и `end`.

Параметр sep

Давайте рассмотрим следующий код:

```
print('aa', 'bb', 'cc')
```

Результатом выполнения такого кода будет:

```
aa bb cc
```

Как вы можете заметить, все строки выводятся с пробелом между друг другом – это неслучайное поведение. У команды `print()` есть параметр, который отвечает за разделение аргументов при выводе. Этот параметр называется `sep` (separator – разделитель). По умолчанию этот параметр равен символу пробела `' '`. Следующие строки кода являются эквивалентными:

```
print('aa', 'bb', 'cc')
```

```
print('aa', 'bb', 'cc', sep=' ')
```

Мы можем изменить параметр `sep` на любую другую строку, например, на символ звёздочки `*`.

Приведённый ниже код:

```
print('aa', 'bb', 'cc', sep='*')
```

ВЫВОДИТ:

```
aa*bb*cc
```

Сейчас у нас аргументы разделены символом звёздочки `*`. Также в качестве параметра `sep` мы можем указать и переменную.

Приведённый ниже код:

```
minus = '-'  
print('aa', 'bb', 'cc', sep=minus)
```

ВЫВОДИТ:

```
aa-bb-cc
```



Таким образом, необязательный параметр `sep` команды `print()` позволяет установить строку, с помощью которой будут разделены аргументы (если их несколько) при печати.

Параметр `end`

Теперь давайте рассмотрим ситуацию, когда у нас не один `print()`, а несколько.

Приведённый ниже код:

```
print("A great man doesn't seek to lead.")
print("He's called to it. And he answers.")
```

ВЫВОДИТ:

```
A great man doesn't seek to lead.
He's called to it. And he answers.
```

Как вы можете заметить, после каждого `print()` курсор переходит на новую строку. И это поведение тоже не является случайным, потому что у команды `print()` есть параметр `end`, определяющий, что нужно добавить в конец вывода. По умолчанию параметр `end` равен символу перевода строки (`\n`).

Следующие строки кода являются эквивалентными:

```
print("A great man doesn't seek to lead.")
print("He's called to it. And he answers.")
```

```
print("A great man doesn't seek to lead.", end='\n')
print("He's called to it. And he answers.", end='\n')
```

Если перевод строки делать не нужно или требуется указать специальное окончание для вывода, то следует явно указать значение для параметра `end` (можем указать через переменную, как и с параметром `sep`).

Приведенный ниже код:

```
minus = '-'
print('a', 'b', 'c', end=minus)
print('second line')
```

ВЫВОДИТ:

```
a b c-second line
```

По завершении печати первой команды `print()` вставлен символ `-` вместо символа перевода строки `\n`.



Таким образом, параметр `end` определяет строку, которая будет добавлена после вывода всех аргументов команды `print()`.

Примечания

Примечание 1. Вызов команды `print()` с пустыми скобками делает перевод строки.

Приведённый ниже код:

```
print('Python')
print()
print('C#')
print('Java')
print()
print('JavaScript')
```

ВЫВОДИТ:

```
Python

C#
Java

JavaScript
```

Примечание 2. Последовательность символов `\n` называется **управляющей последовательностью** и задаёт перевод строки.

Приведённый ниже код:

```
print('a', '\n', 'b', '\n', 'c', sep='*', end='#')
```

ВЫВОДИТ:

```
a*
*b*
*c#
```

Примечание 3. Параметры `sep` и `end` можно устанавливать одновременно.

Приведённый ниже код:

```
print('a', 'b', 'c', sep='*', end='finish')
```

ВЫВОДИТ:

```
a*b*cfinish
```


Примечание 4. Для разных команд `print()` можно задавать разные параметры `sep` и `end`.

Приведённый ниже код:

```
arg1 = 'Hello'
sep1 = '_ _'
end2 = '+++'

print(arg1, 'everyone', sep=sep1, end='! ')
print('How', 'are', 'you', 'in', '2024?', sep=' ', end=end2)
```

ВЫВОДИТ:

```
Hello_ _everyone! How are you in 2024?+++
```

Примечание 5. Чтобы убрать все дополнительные выводимые символы, можно установить параметры `sep` и `end` команды `print()` как пустые строки (`''`).

Приведённый ниже код:

```
print('a', 'b', 'c', sep='', end='')
print('d', 'e', 'f', sep='', end='')
```

ВЫВОДИТ:

```
abcdef
```

Примечание 6. Если после вывода данных нужно более одного перевода строки, то можно использовать следующий код:

```
print('Python', end='\n\n\n')
```

Примечание 7. Мы не можем указывать параметры `sep` и `end` перед аргументами, так как именованные параметры всегда должны следовать после позиционных аргументов. Подробнее эту тему мы разбираем в курсе для продвинутых, сейчас же рекомендуем вам просто запомнить.

Приведённый ниже код:

```
print('5', '+', sep='_', '5', end='!!!', '=', '10')
```

приводит к возникновению ошибки:

```
SyntaxError: positional argument follows keyword argument
```

Примечание 8. Параметр `sep` является разделителем для нескольких аргументов в команде `print()`.

Если аргумент в команде `print()` только один, то параметру `sep` нечего разделять. В таком случае параметр `sep` никак не будет влиять на выводимый текст.

Приведённый ниже код:

```
print('Python', sep='777')
```

ВЫВОДИТ:

```
Python
```

Множественное присваивание

В языке Python за одну инструкцию присваивания можно задавать значения сразу нескольким переменным. Делается это так:

```
name, surname = 'Timur', 'Guev'  
print('Имя:', name, 'Фамилия:', surname)
```

Этот код можно записать и так:

```
name = 'Timur'  
surname = 'Guev'  
print('Имя:', name, 'Фамилия:', surname)
```

Отличие двух способов состоит в том, что множественное присваивание в первом способе присваивает значения двум переменным одновременно.

Если требуется считать текст с клавиатуры и присвоить его в качестве значения переменным, то можно написать так:

```
name, surname = input(), input()  
print('Имя:', name, 'Фамилия:', surname)
```

Если слева от знака `=` в множественном присваивании должны стоять через запятую имена переменных, то справа могут стоять произвольные выражения, разделенные запятыми. Главное, чтобы слева и справа от знака присваивания было одинаковое число элементов.

Множественное присваивание удобно использовать, когда нужно обменять значения двух переменных. В Python это делается так:

```
name1 = 'Timur'  
name2 = 'Gvido'  
  
name1, name2 = name2, name1
```

Обратите внимание, что для обмена значений переменных следующий вариант не сработает:

```
name1 = 'Timur'
name2 = 'Gvido'

name1 = name2
name2 = name1
```

Дело в том, что инструкция `name1 = name2` полностью стирает старое значение переменной `name1`. Когда мы в инструкции `name2 = name1` присваиваем для переменной `name2` значение переменной `name1`, этим значением уже не является строка `'Timur'`, этим значением уже является строка `'Gvido'`.

Примечания

Примечание 1. Названия переменных ничего не говорят интерпретатору о значениях в этих переменных, и даже в очень хорошо названной переменной не появится нужное значение, если мы сами его туда не запишем.

Примечание 2. Новое значение переменной вытесняет старое. Важно представлять, чему равно значение переменной в каждый момент времени.

Примечание 3. В качестве названия переменных запрещено использовать ключевые (зарезервированные) слова. К ключевым словам в языке Python относятся:

1. False;
2. True;
3. None;
4. and;
5. with;
6. as;
7. assert;
8. break;
9. class;
10. continue;
11. def;
12. del;
13. elif;
14. else;
15. except;
16. finally;
17. try;
18. for;
19. from;
20. global;



- 21. if;
- 22. import;
- 23. in;
- 24. is;
- 25. lambda;
- 26. nonlocal;
- 27. not;
- 28. or;
- 29. pass;
- 30. raise;
- 31. return;
- 32. while;
- 33. yield.